PLaND Path to Least Non Determinism

Maddipatla Naga Venkata Sai Krishna, **Asit Kumar Sahoo**

Affiliations: Both authors are Independent Researchers, San Francisco, CA, USA. Corresponding author: Maddipatla Naga Venkata Sai Krishna; mnvsk97@gmail.com.

Abstract

Business processes and agentic systems contain decisions with different computational requirements. Some require interpretation, contextual judgment, novelty handling, or exception resolution; others are stable enough to execute deterministically. When these boundaries are not known in advance, a natural-language agent provides an expressive starting point, but repeatedly routing stable work through a language model creates avoidable model calls and token consumption. We present Path to Least Non Determinism (PLaND), an evaluation-driven methodology for progressively reducing model-mediated computation within accuracy limits set before evaluation. PLaND begins with an entirely English standard operating procedure (SOP). Its evolver skill instructs a host reasoning agent to inspect development examples and execution records, then propose one revised SOP, called a candidate. A candidate may add Python or Bash code and retain model reasoning for unresolved inputs. Evaluation compares predicted and expected answers on separate examples. We compare fixed English-only and hybrid SOPs on LEDGAR legal clauses, CFPB consumer complaints, and SpamAssassin email. The reported results cover 1,000 LEDGAR test clauses and 100 validation examples each for CFPB and SpamAssassin. Acceptance requires both workflows to reach 80% accuracy, an accuracy decrease within two percentage points after accounting for sampling uncertainty, and at least 5% fewer tokens with evidence of a positive reduction. On LEDGAR, accuracy changed from 93.5% to 92.7%, model calls fell from 1,000 to 589, and token use fell 40.02%. LEDGAR met these requirements; CFPB and SpamAssassin did not. Three additional paired runs per dataset reproduced these decisions using the same examples. The experiments evaluate the execution of fixed SOPs; they do not independently evaluate autonomous rule discovery from run histories.

Keywords: agentic workflows, agent skills, SOP evolution, deterministic execution, language-model agents, token efficiency, hybrid systems

1 Introduction

Business processes often contain both stable and ambiguous decisions. A language model can interpret unfamiliar inputs and handle exceptions, but it may also spend tokens repeatedly applying the same recognizable rule. For example, a contract clause that explicitly specifies its governing law may be easier to classify than a clause that combines several legal functions. The practical question is: Which decisions can be handled by code while keeping the workflow's quality within an acceptable range?

Existing infrastructure provides ways to package and execute these workflows. The Agent Skills specification organizes reusable instructions in a `SKILL.md` file with optional supporting files [1]. DeepAgents supports loading skills [2], while LangGraph provides graph-based workflow orchestration [3]. These are existing systems on which a methodology can build, not components introduced or independently benchmarked in this paper.

Related research optimizes model programs, prompts, workflows, and reusable skills. DSPy optimizes language-model pipelines against task metrics [4]. AutoFlow and AFlow generate and improve workflows [5, 6], while Automated Design of Agentic Systems searches over agent designs [7]. SkillOpt, SkillRevise, SkillReducer, and ACES study the optimization or evaluation of reusable skills [8-11]. PLaND focuses on a specific change within a workflow: replacing suitable model-mediated decisions with explicit computation, while retaining model reasoning for the remaining inputs and evaluating the complete result.

PLaND begins with an English standard operating procedure (SOP). We call this initial workflow the baseline: the language model follows the English instructions for every case. A hybrid SOP adds an executable step, meaning code that performs a defined operation when its conditions are met. In our experiments, this step is a Python classification script; cases it cannot classify go to the same language model. We call that path fallback. Figure 1 illustrates these two workflows. Further changes may organize stable steps as nodes in a workflow graph. That is a proposed direction. In this paper, reducing non-determinism means reducing dependence on model-mediated execution. It does not mean finding a globally optimal workflow or proving that repeated model outputs become less variable.

We evaluate this approach on LEDGAR, CFPB, and SpamAssassin. These tasks let us test whether code can avoid model calls, whether accuracy remains within a stated tolerance, and whether the same acceptance decision holds across repeated runs. They are individual classification tasks, not complete business processes.

2 Material and Methods

2.1 Skill structure and executable steps

A skill is a directory whose main instructions are written in `SKILL.md`. Optional `references/` files provide supporting instructions or domain material, `scripts/` contains executable code, and `assets/` holds reusable resources such as document templates, images, or lookup tables [1]. A reference file in this directory is supporting material for the agent; it is not a bibliographic citation and does not make a decision deterministic by itself.

An executable step may use Python or Bash and must have defined inputs and outputs. The workflow must actually execute the code and use its result. For example, a Python classification script can return a label directly.

Merely mentioning that script in the English instructions does not establish that it ran.

Figure 1 illustrates the progression using LEDGAR. The English SOP asks the model to read a clause, identify its legal function, compare the allowed labels, and choose one. The hybrid SOP adds a rule-based classifier. Inputs that the rules cannot resolve return to the model. Section 3.2 explains how the benchmark executes this choice.

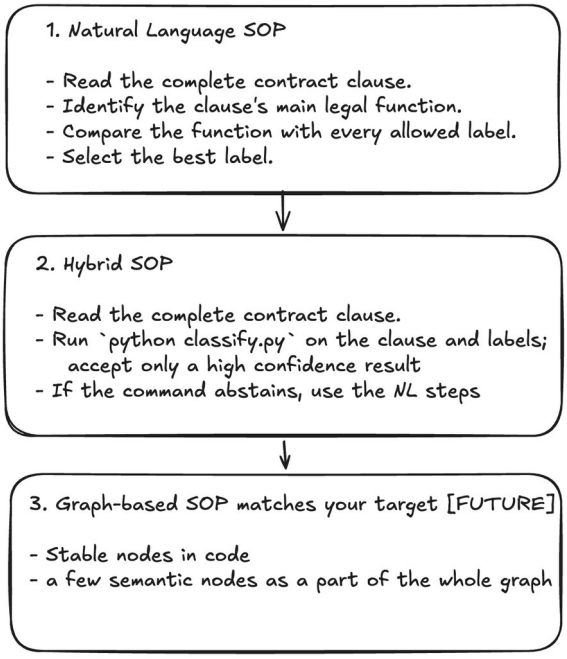

Figure 1. From English instructions to selective code execution. The first two stages summarize the evaluated LEDGAR SOPs. Production logs may reveal recurring paths that can be encoded as steps in a workflow graph, using existing orchestration tools [3]. The third stage illustrates this future direction; it was not evaluated here.

2.2 How an SOP evolves

PLaND uses two skills [12]. The `generate-initial-version` skill creates an initial agent with one English SOP from the task requirements, approved data sources, and evaluation specification. The `pland-evolver` skill then instructs a host reasoning agent to run that SOP, inspect its execution records, and propose changes. The two skills are reusable across tasks; the generated SOP and its code are specific to the task. Appendix D summarizes their instructions and links to the complete skill files.

A candidate is a proposed revised version of the whole SOP package: its instructions and any supporting code. Each candidate contains one bounded change, such as adding a Python classification rule. Search means proposing and evaluating candidates until one meets the requirements or the attempt, time, or cost limit is reached. Evolution is the resulting progression from the current SOP to an accepted revision. A dataset row is one input on which a candidate runs; it is not itself a candidate.

Development, validation, and test are three separate groups of examples from a dataset. Development examples are available while creating and revising the SOP. Validation examples check whether a proposed revision works on cases that did not guide it. The test set stays untouched during selection and provides the final assessment afterward. This separation reduces the risk of accepting a rule that only works on examples already studied. Section 3.1 states the sizes and which evaluations were completed.

The evolution process is:

1. Run the current SOP on development examples. Save its answers, errors, model calls, tokens, and execution records.
2. Inspect those records for recurring patterns or avoidable model work. Propose one change and save the resulting candidate SOP package.
3. Check the candidate on development examples. If it meets the preliminary requirements, compare it with the baseline on separate validation examples.
4. Accept the candidate only if every quality and token requirement in Section 3.3 passes. Otherwise retain the current SOP. To try another revision, return to development evidence and evaluate the new candidate on an unused set. Do not turn validation answers into new rules.
5. Freeze the selected candidate before opening the reserved test set. Run the final assessment without further tuning. Repeated runs of an unchanged package measure repeatability, not another step of evolution.

Consider this illustrative LEDGAR clause: "This Agreement shall be governed by the laws of the State of California." Its expected label is Governing Laws. The baseline asks the model to read the clause and choose a label. Suppose development records show that phrases such as "governed by the laws of" recur in this category. A proposed candidate adds a Python rule for that pattern and uses the model when no single category is identified. The revised instructions and Python rule together form one candidate.

Success on this one clause is insufficient. The candidate and baseline must process many separate validation clauses, and their predicted labels are compared with the expected answers. A narrower phrase rule would constitute another candidate. This example explains how a proposal could arise; it is not a reconstruction of an independently recorded agent discovery. The reported experiments evaluate saved SOPs after their rules were created. They measure accuracy and model use, but do not test whether an agent can discover those rules automatically.

The runner is the program that executes an SOP on each example. The scorer checks the result. For all three classification tasks here, the scorer gives one point when the returned label exactly matches the expected label and zero otherwise. Accuracy is the proportion of correct labels. Expected answers are used for scoring and are not included in the inputs sent to the model or Python classification script. Section 3.2 explains execution, and Appendix B gives the implementation details.

Figure 2 shows what stays fixed during a comparison. Only the SOP package and its directly related files may

change. The task, model, system prompt, selected data, scorer, runtime settings, permissions, and acceptance requirements stay the same for baseline and candidate. File fingerprints and case identifiers check this consistency. The default search budget allows at most ten candidate attempts; it is a stopping limit, not a claim that these experiments generated ten candidates. Appendix D distinguishes that policy from the saved comparisons.

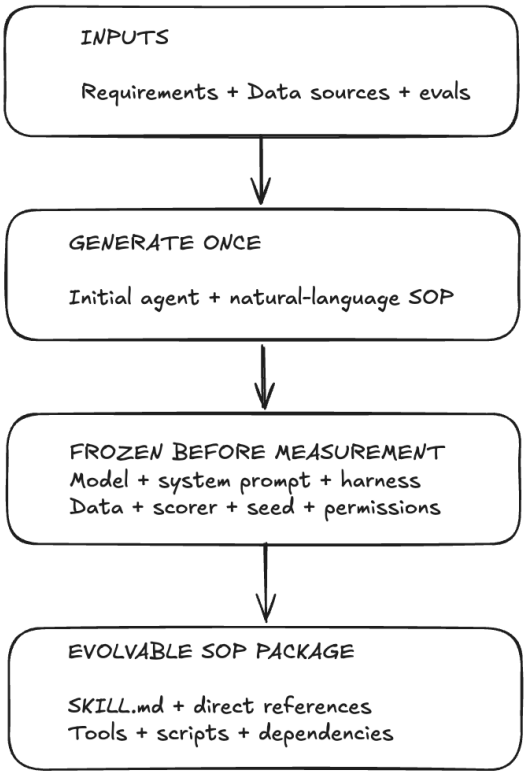

Figure 2. What can change during a comparison. Generate the initial agent and English SOP once, then fix the evaluation setup. Candidate changes are confined to the SOP package. Development records can guide revisions; validation and test answers remain with the evaluator.

2.3 Quality and expense

The quality requirements specify how accurate a workflow must be before lower model use counts as an improvement. For these experiments, both the baseline and candidate must classify at least 80% of cases correctly, and the candidate may lose at most two percentage points of accuracy after accounting for sampling uncertainty. A change from 93.5% to 91.5% is a two-point decrease; it is not a 2% relative decrease. Section 3.3 states the four acceptance requirements, including a minimum 5% token reduction.

Let W denote the baseline workflow and W′ a proposed replacement. Q(W) is classification accuracy, E(W) is total model input and output tokens, Qmin is the minimum acceptable accuracy, and ε is the largest allowed accuracy decrease. Here Qmin = 0.80 and ε = 0.02.

The intended comparison is:

$$E(W') < E(W)$$
$$Q(W) \geq Q_{min},\ Q(W') \geq Q_{min}$$
$$Q(W') - Q(W) \geq -\varepsilon$$

These values are study-specific engineering choices recorded in the comparison configuration [12]. They are not thresholds established by the dataset creators or universal standards for these tasks. The records do not provide an application-specific error-cost analysis that would justify 80% or two points for deployment. We therefore interpret acceptance only under these stated tolerances. A real application should choose its accuracy requirements before evaluation, based on the consequences of its errors. Non-inferiority testing provides a framework for assessing an allowed loss [13]; it does not supply the numerical margin used here.

3 Experimental Setup

3.1 Datasets and data splits

We use three public sources: LEDGAR contract clauses through the LexGLUE task [14, 15], consumer complaint narratives from the Consumer Financial Protection Bureau (CFPB) [16], and the SpamAssassin public email corpus [17]. The tasks are to identify a legal clause type, a complaint's product category, or whether an email is spam. LEDGAR and CFPB each use ten labels; SpamAssassin uses two. Appendix C lists the labels and preparation details.

Table 1. Prepared dataset splits (numbers of examples)

Dataset	Dev.	Validation	Reserved test
LEDGAR	100	100	1,000
CFPB	100	100	1,000
SpamAssassin	100	100	1,000

In Table 1, Dev. means development. Each dataset has three non-overlapping groups, with the roles defined in Section 2.2: development guides revisions, validation selects the candidate, and the reserved test measures the selected version. The table shows examples prepared, not the number of completed model evaluations. These subsets contain equal numbers per label and do not represent natural label frequencies in production. The dataset manifests and audit records identify the selected examples and check for duplicate identifiers, duplicate content, and overlap with earlier development material [12].

In the completed evaluations reported below, LEDGAR passed validation and proceeded to its 1,000-case test. CFPB and SpamAssassin failed validation, so their 1,000-case tests remained unused. Their reported results therefore cover 100 validation examples each. The unequal reported sample sizes follow from this stopping rule; all three datasets had the same split sizes prepared. Appendix C records the evaluation stage for each.

3.2 How the baseline and hybrid classify each case

The baseline and hybrid process the same examples. As defined in Section 1, the baseline sends every case to the language model with the English SOP and the list of possible labels. For the illustrative governing-law clause in Section 2.2, the model reads the clause and chooses Governing Laws.

The hybrid tries the Python classification script first. When a rule identifies one category, the workflow uses that label without calling the model. When the rules find no answer or conflicting categories, the script returns no answer and the workflow sends the same case to the model. This is fallback. For example, a clear governing-law phrase can be handled by code, while a clause matching several categories goes to the model for interpretation. The email script uses the same principle but returns only a spam label when its rule matches.

The scorer checks every final label against the expected answer, regardless of which path produced it. A matching label scores one and any other answer scores zero. Model-token use is the sum of the input and output tokens reported by the model across all cases. A case answered entirely by the Python script uses no model tokens. Appendix B describes the code calls, output checks, and recorded fields.

The evaluated hybrids also changed the English instructions used on the fallback path. In LEDGAR those instructions became shorter when the hybrid was created; shortening was not a separately evaluated architectural decision. The results therefore measure the complete revised SOP, including both its rules and its changed wording. Keeping the original English wording unchanged in another comparison would be necessary to measure the effect of adding code alone. Section 4.1 separates the observed token savings by path.

3.3 Acceptance criteria and uncertainty

We fixed four acceptance requirements in the study protocol before the reported evaluations. The same requirements apply to all three datasets [12]:

1. Minimum accuracy: both workflows must classify at least 80% of examples correctly.
2. Accuracy safety check: after allowing for uncertainty in the sampled examples, the candidate's estimated accuracy loss must remain within two percentage points.
3. Token saving: the candidate must use at least 5% fewer total model tokens.
4. Token safety check: the uncertainty range for the token saving must stay above zero.

All four must pass. The accuracy and token thresholds are the authors' study settings, as explained in Section 2.3. They do not come from the dataset creators and do not establish deployment requirements for a particular application.

An uncertainty range accounts for the fact that a different sample of examples could give a different result. For LEDGAR, the candidate scored 92.7% against the baseline's 93.5%, a decrease of 0.8 percentage points. Its estimated range ran from 1.6 points worse to no change. The less favorable end, 1.6 points worse, was still within the two-point allowance. This passed the accuracy safety check; it does not guarantee that accuracy can never fall by more than two points.

Formally, we calculate a paired 95% bootstrap interval [18]. "Paired" means both workflows process the same cases. Its lower endpoint must be at least -2 percentage points for accuracy change and strictly above zero for token reduction. Appendix B explains the calculation. These intervals describe uncertainty from sampling examples within the prepared task; they do not cover changes in the model or production data.

Figure 3 shows where the acceptance decision belongs in the evolution process described in Section 2.2. The current paper reports one fixed baseline and hybrid comparison per dataset, with three further paired runs described in Section 3.4.

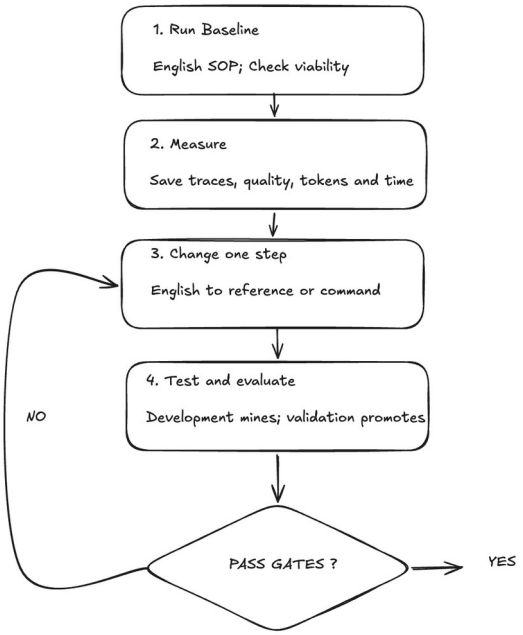

Figure 3. How a candidate is proposed and checked. A candidate is one revised SOP package. Search repeats the proposal and evaluation steps within the attempt limit. Development examples guide changes; separate validation examples decide acceptance. "Pass gates" means meeting all four requirements in Section 3.3. After the Yes branch, the selected candidate is frozen for its reserved test. Revising a rejected candidate requires development evidence and an unused evaluation set.

3.4 Execution environment and repeated runs

The experiments used `qwen3:14b` through local Ollama on an Apple M5 Pro with 48 GB unified memory. The main comparisons processed one case at a time. Appendix B records the model settings and implementation details.

We then repeated the baseline and hybrid comparison three times for each dataset. One paired run means that both workflows process the same evaluation examples under matching model and runtime settings. The three runs reused the 1,000 LEDGAR examples and the 100

examples each for CFPB and SpamAssassin described in Section 3.1. Repeating a 1,000-case evaluation three times still covers 1,000 distinct cases. It measures repeatability, and does not create three new dataset splits or three new candidate SOPs.

For these repeated runs, the local setup allowed at most two requests at the same time. This was an execution constraint, not a requirement of PLaND. The repeats also used a stricter output format, so Appendix A reports them separately from the original comparisons. Appendix B explains all settings, including inference seeds.

4 Results

Table 2 presents the main results. All accuracy and token comparisons show the natural-language baseline first and the hybrid second. A pass means that all four criteria in Section 3.3 were met; it does not mean that accuracy was identical.

Table 2. Main evaluation results

Dataset	Baseline to hybrid result
LEDGAR, 1,000 cases	Accuracy: 93.5% → 92.7%; tokens: 376,088 → 225,573 (40.02% reduction); Pass
CFPB, 100 cases	Accuracy: 79.0% → 72.0%; tokens: 60,514 → 35,247 (41.75% reduction); Reject
SpamAssassin, 100 cases	Accuracy: 90.0% → 86.0%; tokens: 238,077 → 228,359 (4.08% reduction); Reject

Table 3 supplies the uncertainty estimates used for these decisions. Accuracy changes and their intervals are in percentage points. Token-reduction intervals are percentages of baseline token use.

Table 3. Paired uncertainty estimates for the main comparisons

Dataset	Observed change and paired 95% interval
LEDGAR	Accuracy: −0.8 points (−1.6 to 0.0); token reduction: 40.02% (37.00% to 43.08%)
CFPB	Accuracy: −7.0 points (−13.0 to −2.0); token reduction: 41.75% (31.30% to 53.14%)
SpamAssassin	Accuracy: −4.0 points (−9.0 to +1.0); token reduction: 4.08% (1.02% to 8.50%)

4.1 LEDGAR

On the 1,000 LEDGAR clauses, the baseline classified 935 cases correctly and the hybrid classified 927 correctly: eight fewer correct answers, or a 0.8-percentage-point decrease. The paired interval ranged from a 1.6-point decrease to no change. Its lower end remained within the study's two-point tolerance, so the quality criterion passed. This result supports acceptance under that tolerance, not a claim that quality was unchanged.

The hybrid handled 411 clauses through the Python classification script and sent the remaining 589 to the model. It therefore removed 41.1% of model calls and reduced tokens from 376,088 to 225,573, a 40.02% decrease. The Python script classified 395 of its 411 clauses correctly, giving measured accuracy of 96.11% on those cases. Although the script attaches a fixed confidence value of 0.99 to each match, that number is written into the code. It does not mean that 99% of its answers are correct.

The token records distinguish two locations of savings. The baseline used 146,366 tokens on the 411 clauses that the hybrid handled without the model. On the remaining clauses, tokens decreased from 229,722 to 225,573, a difference of 4,149. Thus, 97.24% of the total 150,515-token saving occurred on bypassed calls, and 2.76% occurred on the fallback cases. The fallback SOP was shorter, but this accounting does not separate the effect of shorter input instructions from changes in generated output. The combined SOP change remains a limitation of attributing the full result to deterministic routing.

4.2 CFPB

CFPB's hybrid reduced model calls from 100 to 58 and tokens by 41.75%, but accuracy fell from 79% to 72%. Both variants were below the 80% minimum, and the seven-point observed decrease also failed the relative-quality requirement. The candidate was rejected and the reserved test was not evaluated. Lower token use did not compensate for the loss in classification quality.

Further development of the complaint SOP and its Python rules might improve the result. Additional tailoring is outside this comparison, so these results do not test that possibility. We evaluate the saved packages under common requirements to keep the assessment consistent across tasks. This does not establish equal development effort or the best achievable accuracy for each task.

4.3 SpamAssassin

SpamAssassin's hybrid reduced model calls from 100 to 97. Accuracy fell from 90% to 86%, and token use fell only 4.08%. It failed both the allowed-loss criterion and the 5% token-reduction target, so the reserved test was not evaluated. The three emails handled by the Python classification script were classified correctly by both variants; the observed accuracy decrease occurred among cases sent to the model. This makes the fallback wording relevant even when the Python rules themselves make no errors on the cases they handle.

More suitable email rules or a revised English SOP might improve both accuracy and model use. As with CFPB, additional tailoring falls outside this comparison and would need separate development and evaluation. The PLaND skills remain task-agnostic; the SOP and classification rules are the parts tailored to the task. We retained the same evaluation requirements rather than tuning a rejected package against its evaluation answers.

5 Discussion

5.1 Model reasoning as a resource

The main systems argument is that non-deterministic reasoning should be allocated where it is needed rather than applied uniformly across a workflow. Business processes often contain both stable and ambiguous decisions. When stable regions are repeatedly routed through a language model, the workflow spends model calls and tokens on work that ordinary computation may be able to execute directly.

PLaND provides a controlled way to move that boundary. The English baseline offers a flexible starting point when the structure of the task is not fully known. Development runs reveal errors, repeated reasoning, and expense. The evolver skill guides a host agent to propose a bounded change. Evaluation then determines whether the resulting workflow meets the chosen requirements. This approach may be useful in domains such as fraud detection, where known patterns can be encoded directly while novel narratives or combinations still require contextual reasoning. Fraud detection itself was not evaluated here.

The useful unit of change is not necessarily an entire workflow step. LEDGAR shows that some inputs to a semantic classification task can be handled by explicit rules, while the remaining inputs still require the model. Contract-clause classification remains a semantic problem overall. Its more recognizable cases nevertheless offer opportunities to avoid model calls, beyond mechanical operations such as counting or schema validation.

The benefit must be considered alongside the error tolerance. LEDGAR saved roughly two fifths of tokens but produced eight additional errors per 1,000 test cases. Whether that tradeoff is acceptable depends on the application. CFPB and SpamAssassin did not satisfy the same evaluation criteria. The three results support evaluating each proposed substitution rather than assuming that fewer model calls imply an acceptable workflow.

5.2 What token savings measure

PLaND's generation instructions ask the agent to consider resource use beyond tokens, including CPU, memory, storage, network access, caching, and setup work. The framework can represent different expense objectives. In the experiments reported here, however, the acceptance decision used model tokens as its only efficiency measure. Model-call counts explain the mechanism; they are not a second independently optimized objective.

Tokens and calls are useful measures of model use, but they do not directly establish dollar, energy, or end-to-end latency savings. Local inference time can depend on model loading, hardware utilization, and concurrency. Code also has execution and maintenance costs. Claims about those costs require corresponding measurements and are outside the present evaluation.

5.3 From classification tasks to business workflows

The retained tasks resemble individual decisions within business processes. They do not include long-running state, user interaction, external side effects, or recovery from partial failure. Extending the result to a complete workflow would require evaluating those behaviors as well as label accuracy. The final test evidence is also limited to LEDGAR, one local model, and a balanced subset of ten labels. The CFPB and SpamAssassin outcomes are validation results, not estimates from their reserved test sets.

Temperature-zero decoding and the small number of repeated runs further limit conclusions about variability. As Appendix A shows, both the baseline and hybrid produced identical labels across the three repeats within each task. This supports repeatability under the tested settings, but does not demonstrate that hybrid execution is less variable than the baseline.

6 Future Work

The next step is to evaluate candidate generation directly. A host agent should receive a recorded set of development runs, propose code under fixed permissions, and have each candidate assessed on unused examples. Repeating that procedure would measure how often the agent finds a useful rule and how much development work it requires. It would separate the ability to discover an improvement from the ability to execute an already saved improvement.

A second direction is to evaluate complete workflows with state, tool calls, and recoverable failures. The scorer would need to check whether the task was completed correctly, not just whether a label matched. Such experiments would test whether local token savings remain useful when deterministic steps interact with a wider process.

A third direction is to study maintenance over time. Input distributions can change, making a previously acceptable rule unreliable. A longitudinal evaluation could test monitoring, rollback, and returning affected inputs to the model. These mechanisms are proposed future work, not features validated by the present experiments.

Runtime evidence could also guide automatic construction of a more deterministic workflow. Production logs may reveal recurring input patterns, repeated tool sequences, failures, and expensive model calls. A future system could use those records to propose a graph in which stable paths execute as code and ambiguous cases retain model reasoning, as illustrated in Figure 1. Such proposals would still need evaluation on separate examples before deployment. Whether an agent can reliably generate and maintain this complete workflow remains an open question.

Further comparisons should include other models, languages, and methods for generating code from examples or execution records. A routing-only comparison that preserves the entire baseline fallback would clarify the contribution of each change. Application-specific error costs and direct resource

measurements would make acceptance decisions more relevant to deployment.

7 Conclusion

PLaND is a methodology for reducing model-mediated work through evaluated changes to an English SOP. Its central mechanism is selective code execution with model fallback. On LEDGAR, the evaluated hybrid reduced model calls by 41.1% and tokens by 40.02%, with accuracy decreasing from 93.5% to 92.7%. It passed the study's stated two-percentage-point tolerance. CFPB and SpamAssassin did not pass validation, and their reserved tests were not evaluated. Three additional paired runs per dataset reproduced those decisions. These results demonstrate the usefulness of testing both quality and token use before accepting a substitution, while leaving automatic rule discovery and complete production workflows for separate evaluation.

Acknowledgements and Disclosures

The authors used AI-assisted tools for coding, experiment support, and manuscript preparation. The authors are responsible for the reported results and the final text. No external funding was received, and the authors declare no competing interests.

Data and Code Availability

The PLaND repository provides the two methodology skills, shared classification runner and scorer, saved SOPs, and evaluation records [12]. The LEDGAR results, CFPB results, and SpamAssassin results links identify a fixed evidence snapshot. Table 2 uses the files with prefix `confirmatory-test` for LEDGAR and `confirmatory-validation` for CFPB and SpamAssassin. Each experiment's `variance-study-20260903` folder contains the three paired runs in Appendix A. Appendix C identifies the dataset manifests and Appendix D links to the skill instructions.

Saved run records include case identifiers, expected and predicted labels, correctness, the path used, model calls, tokens, timing, and file fingerprints. These records support auditing the comparisons; they are not a complete record of autonomous candidate discovery. Raw datasets are subject to their source terms and are not included in the repository. Exact regeneration requires the recorded input files and model version. In particular, the CFPB inputs came from a saved local API snapshot that is not redistributed. A fresh download from the live database would constitute a different input snapshot. Repository verification checks saved files and code tests; it does not rerun model inference. Appendix B gives the verification command and the separate requirements for rerunning an experiment.

Appendix A Repeated runs

The repeatability check introduced in Section 3.4 ran each fixed baseline and hybrid three times on the same prepared cases. Table 4 reports the results separately because the runtime configuration changed. These are repeated measurements of existing evaluation sets, not additional independent test samples.

Table 4. Three additional paired runs per dataset

Dataset	Result in all three runs
LEDGAR, 1,000 cases	Accuracy: 93.7% → 92.9%; mean tokens: 376,090 → 225,575.33 (40.02% reduction); Pass
CFPB, 100 cases	Accuracy: 78.0% → 71.0%; mean tokens: 58,372 → 33,120 (43.26% reduction); Reject
SpamAssassin, 100 cases	Accuracy: 88.0% → 85.0%; mean tokens: 188,384 → 178,880.67 (5.04% reduction); Reject

Within each dataset and variant, predicted labels were identical across the three runs, so the sample standard deviation of accuracy was zero. Baseline tokens were identical across runs. Hybrid tokens ranged from 225,575 to 225,576 for LEDGAR and from 178,880 to 178,881 for SpamAssassin, with sample standard deviation 0.58 tokens in each case; CFPB used 33,120 tokens in every run. Model calls remained 1,000 to 589 for LEDGAR, 100 to 58 for CFPB, and 100 to 97 for SpamAssassin.

LEDGAR's paired accuracy intervals were −1.6 to −0.1, −1.6 to 0.0, and −1.6 to 0.0 percentage points, all within the two-point tolerance. CFPB failed both the absolute and relative accuracy requirements in all three runs. SpamAssassin exceeded the 5% token target in these runs but still failed the allowed-loss requirement. Because both variants had zero cross-run label disagreement, the repeats do not show a reduction in output variability attributable to the hybrid.

Appendix B Reproduction details

B.1 Runtime and scoring

The shared classification runner calls the Python function in `classify.py` directly. The `python classify.py` wording in the saved SOP names the executable step; the benchmark does not ask an autonomous agent to launch a shell process. LEDGAR and CFPB return a label when the text patterns identify exactly one allowed category. SpamAssassin returns spam when its rule matches. A missing or disallowed label sends the case to the model. The fixed 0.99 value returned by the scripts is not a measured confidence or an enforced probability threshold.

The scorer is implemented in the shared classification runner. It compares the returned label with the label stored in the evaluation row. A matching label is correct; a different or missing label is incorrect. The same exact-match rule is used for all three datasets. Expected labels are read by the evaluator but are not included in the model prompt or the Python classifier's arguments. Output parsing errors and timing are recorded separately from label correctness.

The repeated runs used the `qwen3:14b` model on an Apple M5 Pro with 48 GB unified memory. The original runner requested JSON output, disabled thinking and streaming, set temperature to zero, and capped generated

output at 128 tokens. It did not explicitly set a context-window option.

The repeated-run configuration used a 4,096-token context, the same 128-token output cap, and an exact JSON schema for the label and confidence fields. It preloaded and retained the model, enabled Flash Attention and q8_0 KV cache, kept one model loaded, and allowed two parallel requests. Pair order was baseline then hybrid, hybrid then baseline, and baseline then hybrid. Exact settings, commands, timestamps, and environment records accompany the run manifests.

B.2 Seeds and repeated runs

An inference seed sets the starting state of the model runtime's pseudo-random procedure. Using the same seed and settings within a baseline/hybrid pair makes their executions more comparable; it does not guarantee identical outputs across software or hardware changes. The three paired runs used seeds 20260903, 20260904, and 20260905. Dataset selection and statistical resampling have their own seeds. Changing an inference seed reruns the model on the same cases; it does not create new data or a new candidate.

The main comparisons comprise one baseline/hybrid pair per dataset. The three additional pairs per dataset give four pairs per dataset in the archived evidence. Across three datasets, this is twelve pairs, or twenty-four executions of an SOP over its evaluation set. Development runs and the LEDGAR selection check are outside that count.

B.3 Uncertainty calculation

The comparison code sorts records by case identifier and draws 5,000 paired bootstrap resamples [18]. Each resample selects cases with replacement and retains both workflows' results for each selected case. It then recalculates the accuracy difference and token reduction. The 2.5th and 97.5th percentiles form the reported 95% interval. The lower endpoint is the less favorable estimate used in the safety checks in Section 3.3. It is not a guaranteed worst possible outcome or a 95% probability statement about this particular interval.

The token bootstrap uses the comparison seed plus one; this is separate from the model's inference seed. The saved records also include Wilson intervals for individual accuracies and an exact McNemar test, but neither determines acceptance under the four criteria. The frozen protocol records the numerical thresholds and resampling settings.

B.4 Verification and rerunning

From a checkout containing the `reproduce/` directory, `uv sync --project reproduce --frozen` installs the locked test environment, and `reproduce/.venv/bin/python reproduce/verify.py` checks the saved evidence and runs code tests. This verifies repository files without calling the model. The pinned evidence snapshot in reference [12] predates this simplified verification entry point; its experiment folders retain the original run commands. Full reruns require the source files identified by the dataset manifests and the recorded model version. The dataset preparation instructions describe preparation; the repeatability study contains its runner, preflight settings, and results. The unavailable CFPB snapshot limits exact public regeneration as stated in Data and Code Availability.

Appendix C Dataset sources and selection

LEDGAR comes from the contract-clause corpus introduced by Tuggener et al. [14], using the LexGLUE classification task [15]. Our selected labels are Governing Laws, Counterparts, Notices, Entire Agreements, Severability, Amendments, Survival, Assignments, Expenses, and Terms. The preparation retains the source split boundaries: development examples come from the source training split, validation from validation, and reserved test from test. The main reported LEDGAR result uses the reserved test after selection on a separate validation set.

CFPB uses the product field associated with published complaint narratives in a saved Consumer Complaint Database snapshot [16]. The selected categories are Mortgage; Checking or savings account; Student loan; Money transfer, virtual currency, or money service; Vehicle loan or lease; Prepaid card; Payday loan, title loan, personal loan, or advance loan; Credit card; Debt collection; and Credit reporting or other personal consumer reports. These are the labels retained from that snapshot; newer source records may use different categories.

SpamAssassin uses the public email corpus [17], combining legitimate email (ham) from the easy-ham and hard-ham archives with spam from the spam and spam-2 archives. The selected source files are the four archives dated 20030228. CFPB and SpamAssassin did not supply the same predefined split structure as LexGLUE; preparation assigned separate examples from their saved sources to development, validation, and test. Their main results use validation because the candidate did not pass the criteria for opening the reserved test.

Each experiment's `confirmatory-dataset.json` records split sizes, selection seed, source-file hashes, exclusions, and audit results [12]. The dataset proof records retain the checks against prepared inputs. The audit found zero overlapping case identifiers, zero duplicate selected content, and zero overlap with the excluded earlier development examples. All three prepared splits were balanced within the selected labels. These checks concern the saved selections, not the full source datasets.

Appendix D What the two PLaND skills instruct

The generate-initial-version skill specifies the initial agent, one English SOP, approved data access, and task-specific runner and scorer files. It derives the scaffold from requirements and the evaluation structure without copying case answers into the agent. Python or Bash replacements for SOP steps are introduced only during subsequent evolution.

The pland-evolver skill instructs the host agent to measure the current SOP, inspect development records,

propose one bounded change, and compare the candidate under fixed conditions. It saves the hypothesis, package changes, measurements, and accept/reject decision. A rejected candidate leaves the previous accepted version in place. Once a reserved test result is opened, the skill instructs the agent to stop evolving and report it.

The default limit is ten candidate attempts. The search may end earlier when the requirements are satisfied or a configured time, cost, or no-improvement limit is reached. Exhausting the budget is not acceptance. The reported text comparisons contain one fixed baseline package and one fixed hybrid package per dataset; they do not establish how many autonomous proposals were needed to create those packages. Three repeated executions of the same pair therefore do not count as three candidate attempts.

The linked evolver policy requires a newly added executable step to preserve the complete English fallback, with later instruction shortening evaluated separately. The saved hybrids studied here also changed fallback wording, as disclosed in Section 3.2. Their measurements assess those complete packages and do not establish compliance with every instruction of that later policy.

References

[1] Agent Skills. Specification. Official documentation. Accessed September 5, 2026.

[2] LangChain. Skills in DeepAgents. Official documentation. Accessed September 5, 2026.

[3] LangChain. LangGraph overview. Official documentation. Accessed September 5, 2026.

[4] Khattab O, et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. 2023. arXiv:2310.03714.

[5] Li Z, et al. AutoFlow: Automated Workflow Generation for Large Language Model Agents. 2024. arXiv:2407.12821.

[6] Zhang J, et al. AFlow: Automating Agentic Workflow Generation. 2024. arXiv:2410.10762.

[7] Hu S, Lu C, Clune J. Automated Design of Agentic Systems. 2024. arXiv:2408.08435.

[8] Yang Y, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. 2026. arXiv:2605.23904.

[9] Liu Y, et al. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. 2026. arXiv:2606.01139.

[10] Gao Y, et al. SkillReducer: Optimizing LLM Agent Skills for Token Efficiency. 2026. arXiv:2603.29919.

[11] Kevin C, et al. Evaluating Skills, Not Just Agents: Agentic Continuous Evaluation of Skills. 2026. arXiv:2608.20614.

[12] Maddipatla Naga Venkata Sai Krishna, Asit Kumar Sahoo. PLaND experiment records and implementation. 2026. Evidence snapshot f9aa707.

[13] Walker E, Nowacki AS. Understanding Equivalence and Noninferiority Testing. Journal of General Internal Medicine. 2011;26(2):192-196. doi:10.1007/s11606-010-1513-8.

[14] Tuggener D, von Däniken P, Peetz T, Cieliebak M. LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts. Proceedings of LREC. 2020:1235-1241. ACL Anthology.

[15] Chalkidis I, et al. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. Proceedings of ACL. 2022:4310-4330. ACL Anthology.

[16] Consumer Financial Protection Bureau. Consumer Complaint Database. Official dataset. Accessed September 5, 2026.

[17] Apache SpamAssassin. Public mail corpus. Official dataset. Accessed September 5, 2026.

[18] Efron B. Bootstrap Methods: Another Look at the Jackknife. The Annals of Statistics. 1979;7(1):1-26. doi:10.1214/aos/1176344552.